Maritime Digital Twin Simulator for Ops Planning

By

Aidan Ong Zongren (A0251863B)

Supervisors:
A/P Tong Xin
Mr Lawrence Chew

DSA4288 Honours Project in Data Science and Analytics

Department of Statistics and Data Science

National University of Singapore

2025/2026

Summary

Efficient tugboat dispatching is a critical component of port logistics, yet traditional scheduling models often rely on static assumptions that fail to capture the stochastic nature of maritime traffic. This paper presents a Maritime Digital Twin designed to simulate and optimise tugboat allocation under varying traffic scenarios.

The proposed framework employs a novel four-stage simulation pipeline to bridge the gap between theoretical optimisation and operational reality. First, containership vessel arrivals are simulated as a non-homogeneous Poisson process.

Second, a data engineering pipeline is run to predict vessel lengths from gross tonnage, extract coupled containership-tugboat trajectories, and construct tugboat schedules according to towing guidelines. A Cheapest Insertion construction heuristic is used to ensure resource synchronisation and enforce a 3 cable-length safety buffer between vessels.

Third, a Tabu Search metaheuristic is used to optimise tugboat allocations to towing missions. By treating the tugboat fleet as a dynamic, shared resource pool rather than rigid, continuous routes, the Tabu Search actively navigates neighbourhood permutations to escape local optima, driving the schedule toward a global minimum cost while maintaining strict spatiotemporal safety constraints.

Finally, these missions are evaluated against reactive scheduling policies via agent-based simulation in Unity3D. A custom-built simulation engine synchronises the generated data with a geospatial environment, instantiating assets in real-time to visualise the interaction between tug resources and arriving vessels. This end-to-end framework demonstrates how digital twins can be used not only to visualise historical data but to stress-test scheduling logic against the dynamic, spatial, and stochastic constraints of a living port environment.

1 Introduction

Port efficiency relies heavily on the last mile of marine navigation: the coordination of tugboats to escort vessels during arrival. Tugboat scheduling is a discrete optimisation problem where a finite fleet of service vessel resources must be assigned to tow a sequence of customer vessels to their respective berths.

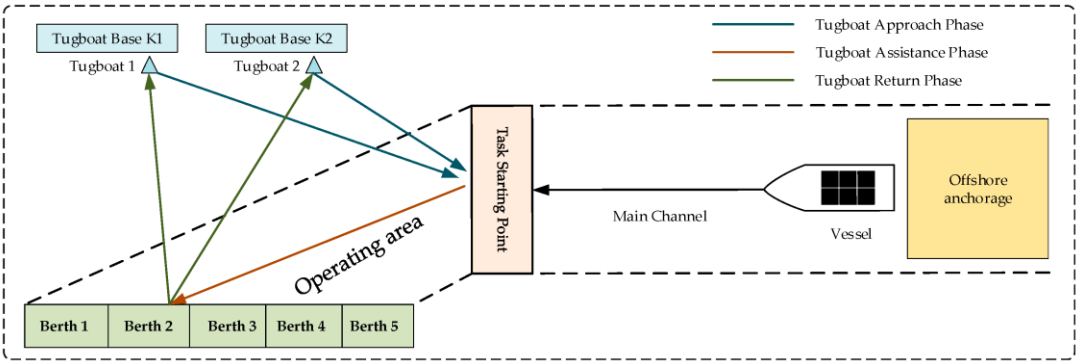

Figure 1: Tugboat Dispatch Diagram

Two types of constraints distinguish tugboat scheduling from other scheduling problems. First, there are **spatial constraints** dictated by the physical topology of the port. Unlike machine scheduling problems where setup times are often negligible or constant, tugboats must traverse significant geographic distances between tasks. The 'inter-job' travel time is strictly dependent on the specific start and end nodes of consecutive assignments (e.g., traveling from the Western Anchorage to Container Terminal 1). Consequently, the availability of a tug is spatially coupled; a resource completing a job at the northern sector of the port is not immediately available for a request in the southern sector.

Second, there are **temporal constraints** governing the execution of service tasks. Each vessel arrival defines a 'release time', i.e. the earliest moment a service can commence. This domain imposes strict synchronisation constraints rarely found in standard logistics: large vessels typically require the simultaneous assistance of multiple tugboats. Consequently, the service

start time is dictated by the arrival of the latest assigned resource, requiring precise coordination across the fleet to avoid idle waiting times.

Despite these complex constraints, current industry practice often relies on manual dispatching or simple heuristics, such as First-Come-First-Served (FCFS), where operators assign resources based on immediate availability rather than global efficiency. This reactive approach often leads to **significant operational inefficiencies,** such as cascading delays during peak traffic hours, and **increases the risk of near-miss events**, where two vessels travelling in congested narrow spaces almost collide with each other. This leads to the following research question:

How can we create and validate tugboat schedules that outperform reactive approaches such as First-Come-First-Served (FCFS), which minimise wait time while maximising safety, across varying traffic scenarios?

To address this question, this paper develops a comprehensive simulation engine to visualise and evaluate tugboat schedule policies. This simulator not only generates feasible schedules that respect all spatial and temporal constraints but actively optimises resource allocation to minimise total vessel waiting time and improve overall port safety. The finalised schedules are then rendered in Unity3D, providing a clear visualisation of port dynamics and the interactions of tugboats with serviced vessels. This simulator can thus be a tool for aiding marine service providers in deploying tugboats optimally to meet regulatory performance requirements.

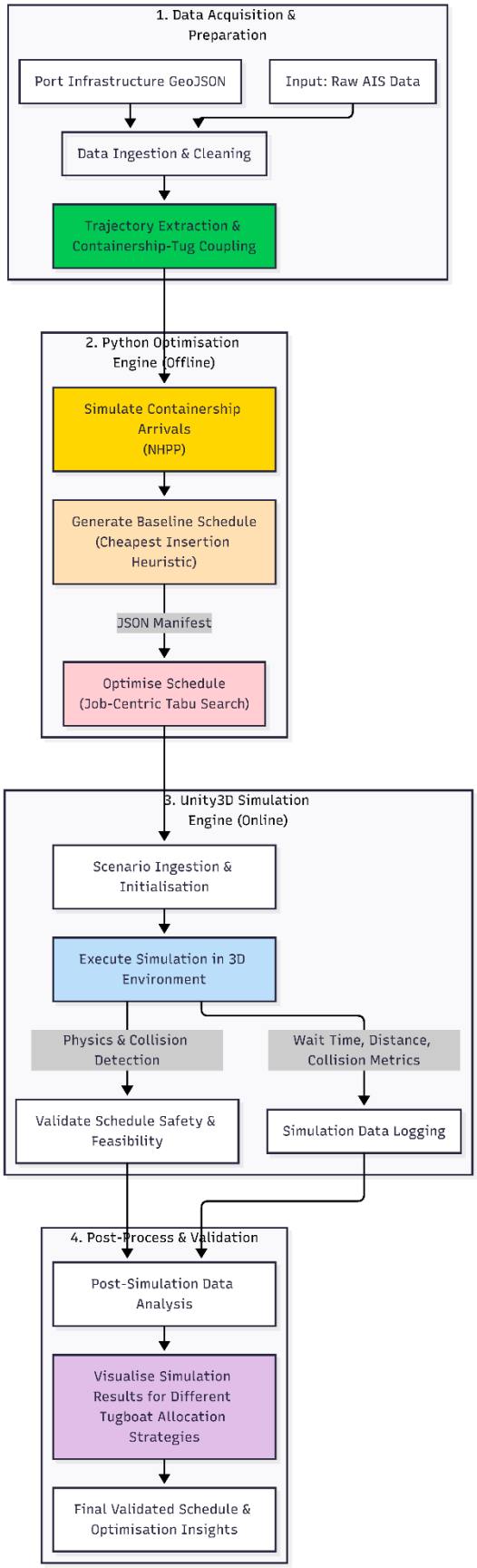

Figure 2: Digital Twin Simulator Flowchart

2 Literature Review

Four papers were reviewed to provide a firm theoretical basis for the paper's methodology.

2.1 Simulation Framework Validation

The decision to model vessel traffic via discrete-event simulation in Unity3D is validated by **Ilati et al. (2014)** in their study on integrated port resource allocation. The group developed a discrete-event simulation model using the **Enterprise Dynamics (ED) platform.** Their approach adopted a discrete berth modelling strategy, representing the continuous quay as distinct allocation slots.

The software utilises simulation blocks called "Atoms" to model real-world entities and store operational logic/data, highlighting the need for object-oriented programming classes to represent complex port assets.

Unity3D mirrors these capabilities through its robust C# scripting environment, which supports Object-Oriented Programming (OOP) principles. Similar to ED's "Atoms," Unity allows for the creation of modular classes (e.g., Vessel, Tug, Berth) that encapsulate specific attributes and behaviours, enabling the precise modelling of complex port interactions.

This object-oriented methodology is further corroborated by **Skaisgiris et al. (2021)** in their development of **PySeidon**, an open-source, data-driven maritime port simulation framework. To manage the complexity of port operations, PySeidon utilises an Entity-Component System (ECS) architectural pattern. In this model, "Entities" serve as behaviour-less labels for simulation objects (such as vessels or terminals), "Components" encapsulate their physical properties and data, and "Systems" provide the operational logic and method implementations. This ECS pattern conceptually parallels Unity3D's native component-based architecture, strongly validating the use of modular, encapsulated C# classes to drive multi-agent maritime interactions.

Furthermore, PySeidon emphasises the critical necessity of grounding these modular simulation frameworks in empirical observation. Unlike traditional simulation models that may assume fixed transit times for port navigation, PySeidon operates as a discrete-time increment simulator that relies heavily on historical Automatic Identification System (AIS) data to define predefined, realistic vessel navigation paths. This data-driven approach directly supports the architectural focus of this digital twin project, validating the methodology of leveraging Unity3D to ingest and simulate real-world AIS trajectories to ensure the resulting port dynamics are both structurally modular and empirically accurate.

However, when handling the assignment of tugboats to vessels, both Ilati and Skaisgiris default to a rudimentary First-Come, First-Served (FCFS) principle. While an FCFS queuing model simplifies the computational overhead of the simulation engine, it fails to reflect the combinatorial complexity of modern multi-tug operations.

This project directly addresses this methodological gap by establishing a digital twin that actively evaluates dynamic tug allocation policies. By moving beyond the static FCFS tug allocation scheme adopted by Ilati and Skaisgiris, this project enables the rigorous comparative analysis of proactive optimisation strategies.

2.2 Data-Driven Fidelity

To ensure the simulation reflects the complexities of port traffic, this paper draws on the methodology of **ShipNaviSim,** a recent data-driven maritime simulator developed by **Pham et al. (2025)**. Pham et al. utilised a large-scale AIS dataset with 59.6 million data points to model vessel movements in high-congestion regions of the Singapore Strait.

Their work validates several key methodological choices for this project:

- **Data Processing:** The use of AIS data to drive "Log-play" agents, strictly following historical trajectories to replicate realistic background traffic density without navigation error.
- **Geometric Constraints:** The modelling of specific "planning regions" or geofenced zones to focus computational resources on critical areas (e.g., crossing points or terminals).
- **Safety Metrics:** The definition of a "Near Miss" event as an approach within 3 cable lengths (555 meters), providing a quantitative benchmark for evaluating schedule safety.

Pham et al. explicitly excluded towed vessels from their study, focusing only on ships "underway using engine". They note that they filtered out "smaller vessels such as tugboats," leaving a significant gap in the literature regarding the simulation of coupled tug-barge dynamics. This project directly addresses this gap by applying similar data-driven fidelity to the specific domain of tugboat operations.

2.3 Optimisation Algorithm Validation

While simulation provides the environment, the optimisation engine requires a solver capable of handling NP-hard scheduling problems in real-time. In their work on dynamic tugboat scheduling, **Wei et al. (2024)** provide a compelling justification for employing **Tabu Search** over exact methods like Mixed-Integer Linear Programming (MILP) or simple heuristics.

In their comparative analysis of task assignment strategies, Wei et al. evaluated a hybrid approach known as Cheapest Insertion with Tabu Search. It consists of two key phases:

Phase I: Cheapest Insertion for Rapid Construction

The first component, **Cheapest Insertion**, functions as a greedy heuristic designed to generate an initial feasible schedule by iteratively inserting unassigned jobs into the existing tug routes at the position that causes the minimal increase in the objective function.

Phase II: Tabu Search for Schedule Improvement

Once the initial schedule is established, the **Tabu Search** algorithm is employed to refine and optimise the task assignments. Through swapping task insertions between two assigned tugboat routes, the Tabu Search effectively navigates the complex solution space by using memory structures (Tabu lists) to escape local optima.

Wei et al.'s advanced allocation strategy, **CI-TS-IR** (which adds an Indifference Zone Selection rollout algorithm), was found to achieve the lowest total sail cost across all task assignment strategies. This literature confirms that Tabu Search is an appropriate and industry-aligned choice for tugboat schedule optimisation.

However, a crucial distinction must be drawn between the approach taken by this project and that by Wei. et al. Wei et al. approached optimisation from a **route-centric** paradigm, treating the challenge as a Vehicle Routing Problem where unfulfilled jobs are sequentially inserted into established tugboat routes.

Conversely, this project implements a **job-centric** paradigm. To better accommodate the dynamic nature of port terminal traffic and the operational reality that massive containerships require multiple tugs simultaneously, **available tugboats are dynamically pooled and assigned to unfulfilled jobs**. By treating the tugboat fleet as an evolving shared resource pool rather than locking individual vessels into rigid, continuous itineraries, the system becomes more adaptable. If a containership is delayed, a job-centric model flexibly reallocates the next best available tugs, preventing the cascading delays that would otherwise shatter a tightly coupled, pre-planned route.

Consequently, our adapted Tabu Search generates neighbourhood permutations not by altering a single tugboat's continuous route, but by swapping specific tugboat assignments across specific, time-bound vessel jobs to minimise cumulative wait times and travel distances.

3 Problem Scenario

3.1 Operational Environment and Experimental Design

This paper focuses on the **Pasir Panjang Terminal**, a high-density container terminal within the Port of Singapore. The primary operational objective is to optimise the allocation and scheduling of a finite fleet of tugboats to assist **containerships**.

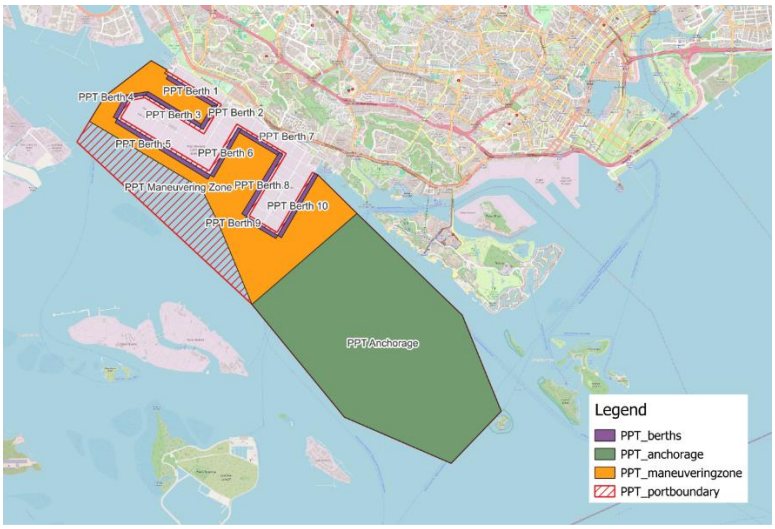

Figure 3: Operational Zones of Pasir Panjang Terminal

The scheduling problem is evaluated using an **Agent-Based Simulation** approach with **Independent Replications**. The experimental design consists of $N = 31$ distinct, independent simulation scenarios, each representing a unique 24-hour period of vessel arrivals.

For each replication, the system generates a complete set of vessel arrival jobs based on a stochastic arrival process. This approach allows for the statistical validation of the scheduling algorithm across diverse operational conditions. The Each job is composed of two coupled operations:

1. **Berthing Operations:** Assisting a vessel from the pilot boarding ground or fairway to its assigned quay.
2. **Unberthing Operations:** Assisting a vessel from the quay back to the open sea or anchorage.

For each scenario, the scheduling workflow proceeds in three stages:

1. **Generation:** Synthetic vessel trajectories and arrival times are generated. These are sorted by ETA to provide a naïve FCFS schedule.
2. **Optimisation:** The Cheapest Insertion and Tabu Search algorithms are executed to produce pre-allocated tug assignment schedules.
3. **Validation:** The schedule is executed in the Unity Digital Twin to measure realised wait times and safety violations.

3.2 Performance Metrics

The performance of the scheduling algorithm is quantified by aggregating results across all simulation runs. The primary Key Performance Indicators (KPIs) are **Mean Average Wait Time** and **Mean Safety Score.**

Mean Average Wait Time is defined as the average wait time per vessel, averaged across all scenarios:

$$\bar{W} = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{1}{M_i}\sum_{j=1}^{M_i}(T_j - ETA_j)\right)$$

Where N is the number of scenarios, T_j is the actual service start time for vessel j, and M_i is the number of vessels in scenario i.

Mean Safety Score aims to measure the frequency and duration of safety buffer violations.

The score for a single run is calculated as:

$$S_i = \frac{1}{1 + \frac{T_{risk}}{3600}}$$

Where T_{risk} is the cumulative duration of risk exposure in seconds.

The final reported metric is the mean score $\bar{S}$ across the N independent runs.

To assess reliability, we also report the standard deviation σ of these metrics. A lower σ indicates that the algorithm performs consistently regardless of daily traffic fluctuations.

3.3 Constraints and Safety Protocols

The generated schedules aim to satisfy two constraints that ensure operational feasibility and safety.

First, large containerships typically require multiple tugs (e.g., 2 to 4) simultaneously. Thus, a strict **Resource Synchronisation Constraint** is imposed, requiring that all assigned tugs arrive at the rendezvous point at the same time. The service cannot commence until the final assigned tug is in position.

Second, to ensure navigational safety, we establish a circular buffer around each active containership vessel. During a scheduled route, a towed vessel that passes within **555 meters** of another moving vessel is considered to have breached the buffer, which decreases the safety of the proposed schedule. Schedules should thus aim to minimise the number of near-miss occurrences, satisfying a soft **Safety Buffer Constraint.**

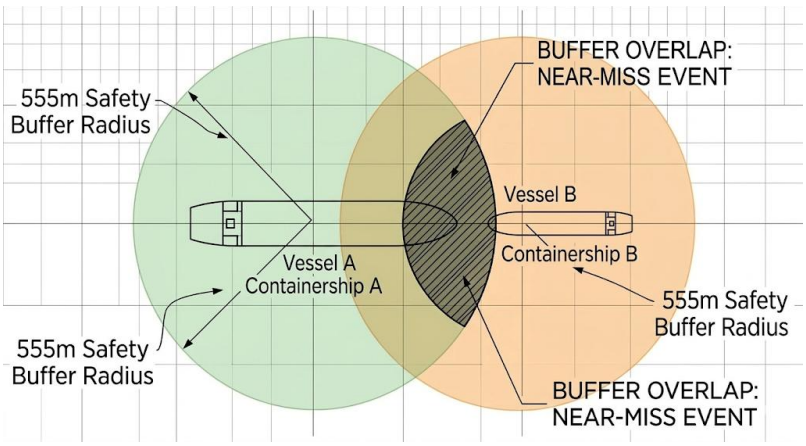

Figure 4: Illustration of safety buffer and near-miss event.

3.4 System Assumptions

To isolate the impact of the scheduling algorithms within the digital twin, the following assumptions are made:

First, the simulation assumes ideal weather, zero tidal drift, and no visibility constraints. Environmental factors that typically affect vessel manoeuvrability and travel times are assumed to be constant.

Second, the simulation applies the 555-meter safety radius universally as a Euclidean distance constraint, regardless of vessel size, movement speed, or the specific physical topography of the port channels.

Third, fleet tugboats are assumed to be instantly available upon reaching their destinations, without the need to account for refuelling, mechanical breakdowns, or crew shift changes. Additionally, tugboats transit between assignments at a constant mobilisation speed of 6.7 m/s (approximately 12 knots), excluding acceleration and deceleration effects.

Finally, we assume a randomised dwelling time of 12 to 24 hours between a vessel's berthing and unberthing operation. These durations are dependent on terminal logistics, such as berth and quay crane availability, which this project will not be considering due to a lack of data. Crucially**, we allow arriving vessels to occupy a berth even when it is being occupied by another dwelling vessel.** Factoring delays due to port congestion into mean average wait time would unfairly penalise the tugboat scheduling heuristics for operational delays that fall entirely outside the scope of navigational fleet management. Moreover, since the berthed vessel is stationary, the safety buffer constraint would not be violated as well. Thus, for simplicity, this project avoids considering the closely related berth allocation problem.

4 Theory

4.1 Modelling Vessel Arrivals

Unlike a standard Homogeneous Poisson Process (HPP) where the arrival rate λ is constant, maritime traffic exhibits distinct temporal patterns due to tidal windows, daylight operations, and supply chain pulses. To capture this, we can model arrivals using an **NHPP**, which is governed by a **time-dependent intensity function** $\lambda(t)$.

Let $\{N(t), t \geq 0\}$ be a counting process representing the number of vessels arriving at the anchorage by time t. The process is an NHPP with intensity function $\lambda(t) \geq 0$ if:

1. $N(0) = 0.$
2. The process has independent increments.
3. The number of arrivals in any interval $(t_a, t_b]$, denoted as $N(t_b) - N(t_a)$, follows a Poisson distribution with mean $\Lambda_{a,b}$

$$P[N(t_b) - N(t_a) = k] = \frac{(\Lambda_{a,b})^k e^{-\Lambda_{a,b}}}{k!}$$

Where $\Lambda_{a,b}$ is the **cumulative intensity**:

$\Lambda_{a,b} = \int_{t_a}^{t_b} \lambda(u)\, du.$

This formulation allows the simulation to generate "bursts" of traffic (high $\lambda(t)$) followed by lulls (low $\lambda(t)$), which is critical for stress-testing tugboat allocation algorithms.

4.2 Parameter Estimation via AIS Data

To ensure the simulation reflects reality, the function $\lambda(t)$ is calibrated using historical Automatic Identification System (AIS) data.

Step 1: Geofencing and Data Extraction

Define a polygon P representing the port boundaries. The dataset can then be spatially filtered to identify unique vessel entry events. An arrival time t_i is recorded when a vessel's coordinates cross the boundary of P. Arrival counts are binned by hour and averaged over a 3-month period.

Step 2: Piecewise Function Step Model

Analysis of average hourly arrival rates at Pasir Panjang Terminal from May to July 2023 revealed distinct temporal patterns driven by vessel type, with Tankers showing the highest arrival volume, followed by Containerships and Bulk Carriers.

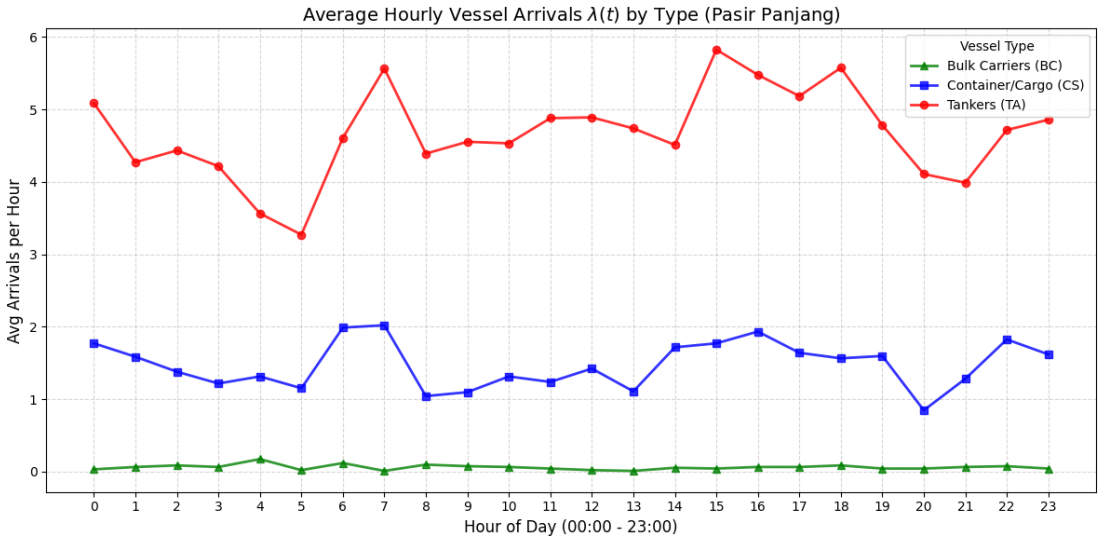

Figure 5: Average Hourly Vessel Arrivals at Pasir Panjang Terminal

A non-parametric Piecewise Constant approximation was selected to preserve the magnitude of peak traffic bursts, which analytical smoothing methods tend to dampen. $\lambda(t)$ is defined as a step function:

$$\lambda(t) = c_i \qquad for\ i \leq\ t < i + 1$$

where c_i is the value from the graph for hour i.

4.3 Tugboat Schedule Problem Formulation

The tugboat scheduling problem is modelled as a resource-constrained optimisation problem where the primary goal is to minimise the aggregate waiting time for all incoming vessels.

Objective Function

Let S_i be the scheduled start time for vessel job i, and x_{ik} be a binary variable equal to 1 if Tug k is assigned to Job i, and 0 otherwise. The objective is the minimisation of **Total Wait Time** (W_{total}), defined as the cumulative difference between the scheduled service start time and the vessel's estimated time of arrival (ETA) at the service gate.

$$\min W_{total} = \sum_{i=1}^{N} (S_i - ETA_i)\ s.t.$$

Resource Synchronisation:

$$S_j \geq (S_i + D_i) + T_{travel}\left(L_{end}^{i}, L_{start}^{j}\right)$$

$$\forall k \in Tugs, \forall\, i, j \in J\ s.t.\ x_{ik} = 1, x_{jk} = 1, i \prec_k j$$

Safety Constraint:

$$\|P_i(t - S_i) - P_{obs}(t)\| \geq \Delta_{safe} \qquad \forall t \in [S_i, S_i + D_i]$$

The first constraint ensures that a tug k cannot be assigned to job j until it has completed its previous job i and travelled to the new rendezvous point. D_i denotes the service duration of job i, T_{travel} is the mobilisation time between the two locations, and $i \prec_k j$ indicating that job i is sequenced immediately before job j in tug k's schedule.

The second constraint imposes a safety buffer, ensuring that the log-played trajectory of vessel i, when shifted to start at time S_i, must not intersect with the trajectory of any dynamic obstacle O (e.g., other moving containerships) at any timestamp t. P_i is the vessel's position relative to its start, P_{obs} is the obstacle's position, and Δ_{safe} is the safety buffer, set at 555m.

4.4 Optimisation Algorithm Implementations

Cheapest Insertion

Let J be the set of unassigned towing jobs, sorted in ascending order by their Estimated Time of Arrival (ETA). The jobs are processed sequentially. For each job $j \in J$, requiring a specific number of tugs R_j, the algorithm identifies the optimal subset of tugboats $K_j^* \subset K$ that minimises the synchronisation time of the required tugboats and the arriving vessel.

For every tugboat k in the total fleet K, we calculate its earliest possible ready time, $t_{ready}(k,j)$, to service job j. This accounts for the time the tug completes its previous assignment $T_{avail}^{(k)}$, the travel distance from its current location $L_{tug}^{(k)}$ to the job's starting location $L_{start}^{(j)}$, and the tug's transit speed v_{tug}:

$$t_{ready}(k,j) = \max\left(T_{avail}^{(k)} + \frac{Dist\left(L_{tug}^{(k)}, L_{start}^{(j)}\right)}{v_{tug}}, ETA_j\right)$$

$$K_j^* = \arg\min_{K_j \subset K, |K_j| = R_j} \left(\max_{k \in K_j} t_{ready}(k,j)\right)$$

Let this optimal, unadjusted synchronisation time be denoted as $t_{synched}^{(j)}$. To ensure that this assignment does not result in collisions with any previously scheduled jobs in the established schedule S, the final predicted start time t_{start} is calculated by introducing a non-negative time shift, $\Delta t \geq 0$, representing the minimum necessary delay to achieve a safe spatial buffer.

$$t_{start} = t_{synched}^{(j)} + \Delta t$$

Let $T_{overlap}$ denote the temporal intersection where both the candidate job j and a previously scheduled job j′ are actively moving within the terminal:

$$T_{overlap} = \left[\max\left(t_{start}, S_{j'}\right), \min\left(t_{start} + D_j, S_{j'} + D_{j'}\right)\right]$$

We implement the safety constraint as a Boolean function that evaluates the Euclidean distance between the interpolated coordinates (x, y) of both vessels at any given overlapping time t:

$$\text{IsTrajectorySafe}(j, t_{start}, S) = \begin{cases} \text{True}, & \text{if } \forall j' \in S, \forall t \in T_{overlap}, \\ & \sqrt{(x_j(t) - x_{j'}(t))^2 + (y_j(t) - y_{j'}(t))^2} \geq \Delta safe \\ \text{False}, & \text{otherwise} \end{cases}$$

Once t_{start} is finalised, the availability times and locations for all assigned tugs $k \in K_j^*$ are updated, and the algorithm proceeds to the next job in J.

Algorithm 1 Safety-Aware Cheapest Insertion Heuristic

Require: List of Jobs J sorted by ETA, Set of Tugs K, Safety Distance d_{safe}
Ensure: Initial Schedule S

```
Initialize tug availability T_avail[k] ← 0 for all k ∈ K
Initialize tug locations L_tug[k] ← BaseLocation for all k ∈ K
S ← ∅
for each job j in J do
    R_j ← required tugs for job j
    Arrivals ← list of tuples (time, tugIndex)
                                        ▷ Step 1: Resource Availability Check
    for each tug k in K do
        t_travel ← Dist(L_tug[k], StartLoc_j)/v_tug
        t_ready ← max(T_avail[k] + t_travel, ETA_j)
        Add (t_ready, k) to Arrivals
    end for
    Sort Arrivals by time ascending
    Candidates ← first R_j tugs from Arrivals
    t_synched ← time of the last tug in Candidates
                                        ▷ Step 2: Spatiotemporal Safety Check
    t_start ← t_synched
    while NOT IsTrajectorySafe(j, t_start, S) do
        t_start ← t_start + Δt                  ▷ Shift forward until safe
    end while
                                                ▷ Step 3: Commitment
    t_end ← t_start + Duration_j
    for each tug k in Candidates do
        T_avail[k] ← t_end
        L_tug[k] ← EndLoc_j
        Assign tug k to job j
    end for
    Set ActualStart_j ← t_start
    Add j to S
end for
return S
```

ETA_u and D_u is determined by the berthing / unberthing trajectory assigned to a tug, which is to be extracted from AIS data. Details of trajectory extraction is covered in **Section 5.** T_{travel} is computed assuming a fixed tug speed $v_{tug} = 6.7$ metres / second, and the distance between locations is calculated using a **RoutePlanner** class. Refer to **Appendix A** for further details.

Tabu Search

Using the schedule generated by Cheapest Insertion as a warm-start solution, the Tabu Search navigates the solution space using the following mechanisms to escape local optima:

Neighbourhood Structure $N(S)$: To generate a candidate neighbour, the algorithm randomly selects a single vessel job, removes one of its currently assigned tugboats, and replaces it with a different tugboat selected from the available fleet. This process is repeated until a sufficient number of candidate neighbours is generated, denoted by N_{size}.

Algorithm 2: Neighborhood Generation via Stochastic Swaps

Input: Current Assignment S_{curr}, Neighborhood Size N_{size}
Output: Set of Candidates N (each containing a schedule and its defining move)

```
N ← ∅
while |N| < N_size do
    Job ← SelectRandomJob()
    Tug_out ← SelectRandomAssignedTug(Job, S_curr)
    Tug_in ← SelectRandomAvailableTug(Job, S_curr)
    move ← (Job, Tug_in)                    // The action taken
    move_rev ← (Job, Tug_out)               // The action to ban
    S_cand ← Copy(S_curr)
    RemoveTug(S_cand, Job, Tug_out)
    AssignTug(S_cand, Job, Tug_in)
    N ← N ∪ {(S_cand, move, move_rev)}
end
return N
```

Evaluation and Selection: The generated neighbours are evaluated against the constraints. Solutions violating the **Safety Constraint** are heavily penalised through adding 100,000 seconds to the wait time. The neighbour producing the lowest W_{total} is selected as the new current solution, even if it is worse than the previous solution (to allow escaping local minima).

Algorithm 3: Candidate Selection with Aspiration Criterion

Input: Candidate Set N, TabuList T, Global Best Cost C_{best}, Current Iteration i
Output: Best Allowed Candidate $(S_{next}, move_{next}, move_{rev}, C_{next})$

```
LowestCost ← ∞
BestCandidate ← Null
foreach (S_cand, move, move_rev) ∈ N do
    C_cand ← EvaluateCost(S_cand)     // Compute Wait Time & Safety
    // Check if the move is currently active on the Tabu List
    IsTabu ← (move ∈ T) and (T[move] > i)
    // The Core Tabu Decision Logic with Aspiration Override
    if ¬IsTabu OR C_cand < C_best then
        if C_cand < LowestCost then
            LowestCost ← C_cand
            BestCandidate ← (S_cand, move, move_rev, C_cand)
        end
    end
end
return BestCandidate
```

Tabu Restriction: To prevent cycling, a move involving job j is declared tabu for a fixed number of iterations (tenure) θ if it reverses a recently performed operation. Let τ_j be the iteration number when job j was last moved. A move is forbidden if:

$$CurrentIter < \tau_j + \theta$$

Algorithm 4: Tabu List Update Mechanism (Expiration Tracking)

Input: TabuList T, Reversed Move $move_{rev}$, Current Iteration i, Tabu Tenure k

Output: Updated TabuList T

```
// Record the iteration when this restriction will expire
ExpirationIteration ← i + k
// Store or update the move in the memory dictionary
T[move_rev] ← ExpirationIteration
return T
```

Aspiration Criteria: The tabu status of a move is overridden if the resulting solution S' has an objective value $W_{total}(S')$ strictly better than the best solution found so far (S_{best}):

$$W_{total}(S') < W_{total}(S_{best})$$

Update Rule: At each iteration, the algorithm moves to the best non-tabu neighbour (or aspiration-satisfying neighbour) $S_{next} \in N(S)$:

$$S_{next} = \arg \min_{S' \in N(S) \setminus Tabu \cup Aspiration} W_{total}(S')$$

Termination: The search terminates after a fixed number of iterations ε to ensure that that the run-time of the algorithm remains feasible.

For the implemented algorithm, the parameters $N_{size} = 20$, $\theta = 15$, and $\varepsilon = 100$ were chosen.

Algorithm 5: Overall Tabu Search Meta-Heuristic

Input: Initial Assignment S_{init}, Max Iterations I_{max}, Tabu Tenure k
Output: Optimized Schedule S_{best}

$S_{curr} \leftarrow S_{init}$
$S_{best} \leftarrow S_{init}$
$C_{best} \leftarrow \text{EvaluateCost}(S_{init})$
$TabuList \leftarrow \emptyset$ `// Dictionary tracking move expiration`
for $i = 1$ **to** I_{max} **do**
  `// Step 1: Generate random neighborhood`
  $N \leftarrow \text{GenerateNeighborhood}(S_{curr}, \text{NeighborhoodSize})$
  `// Step 2: Select best candidate via Cost & Aspiration`
  $(S_{next}, move_{next}, move_{rev}, C_{next}) \leftarrow$
   $\text{SelectCandidate}(N, TabuList, C_{best}, i)$
  `// Step 3: Update state and global best`
  $S_{curr} \leftarrow S_{next}$
  $TabuList \leftarrow \text{UpdateTabuList}(TabuList, move_{rev}, i, k)$
  if $C_{next} < C_{best}$ **then**
    $S_{best} \leftarrow S_{curr}$
    $C_{best} \leftarrow C_{next}$ `// Record new global optimum`
  end
end
return S_{best}

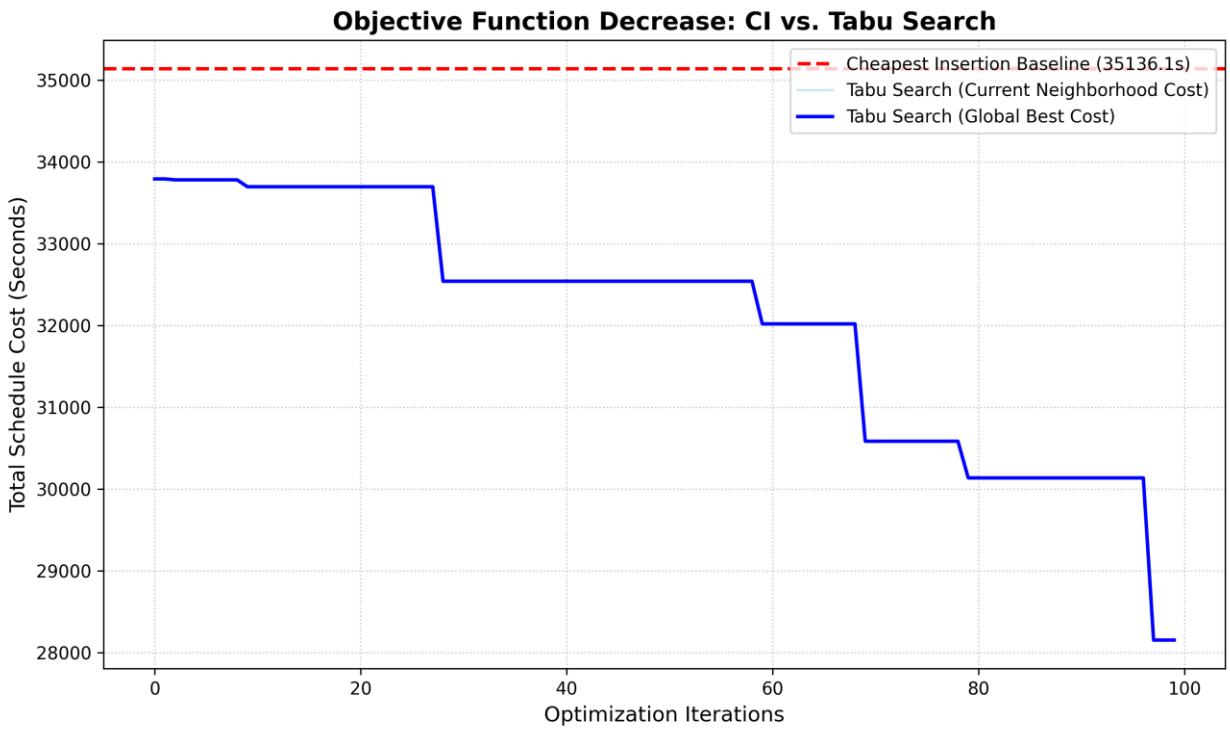

Figure 6: Comparison of Objective Function Values for Cheapest Insertion and Tabu Search.

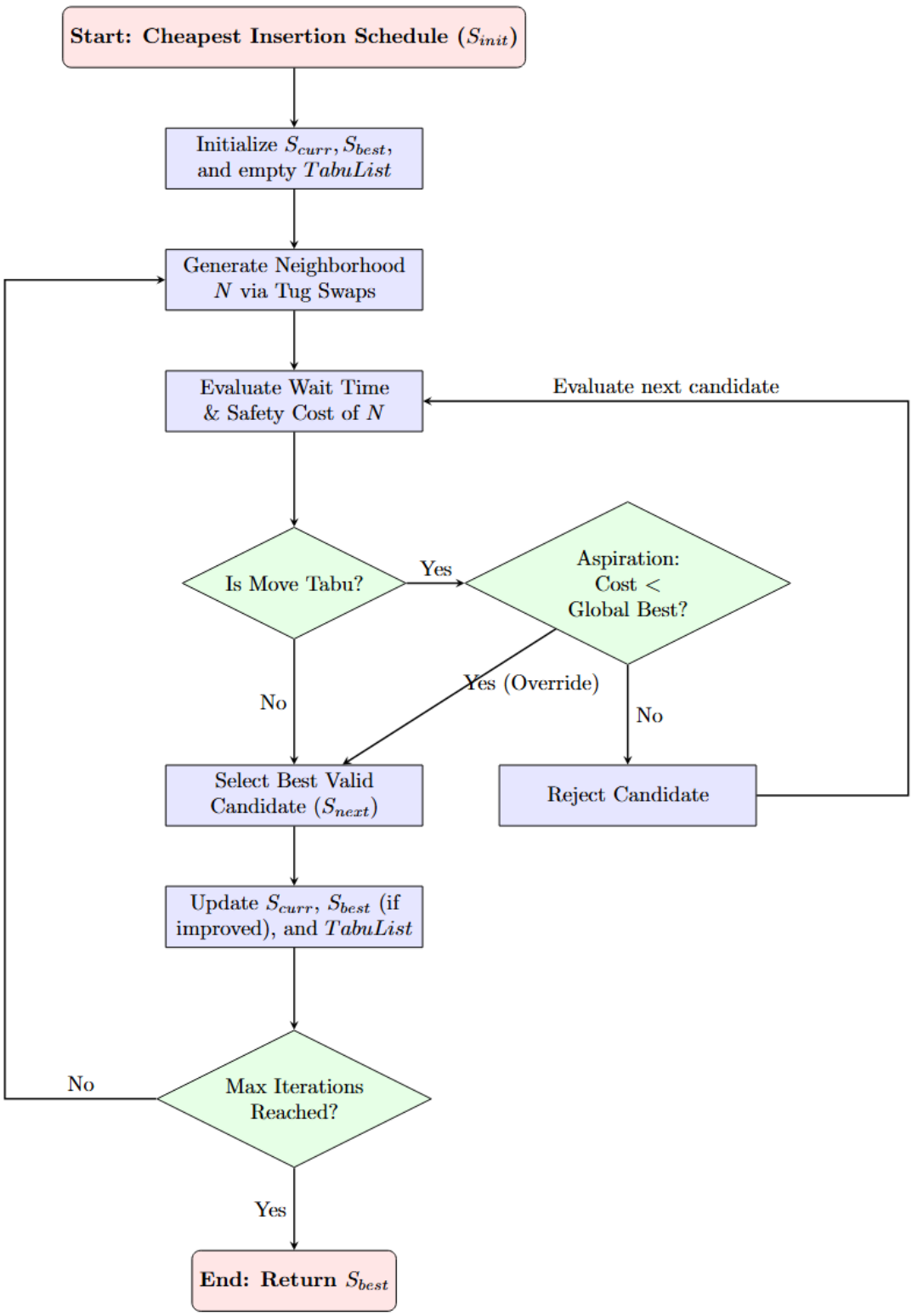

Figure 7: Tabu Search for Tugboat Scheduling Flowchart

4.5 Convergence of Tabu Search

Theoretical foundations exist that establish finite convergence for Tabu Search when it relies on specific recency or frequency memory structures. Glover & Hanafi (2002) showed that under the "Aspiration by Default" rule, tabu search is mathematically guaranteed to systematically visit every solution within a finite domain without infinite cycling. This rule dictates that if the search is stuck (i.e. all immediate neighbours have already been visited), it must move to the neighbour that was visited the least recently or frequently.

However, achieving this mathematical guarantee requires traversing a path whose length scales exponentially with the cardinality of the solution space. As noted in the literature, the sheer magnitude of steps required to satisfy finite convergence can render the guarantee computationally impractical for large-scale applications. In the specific context of generating tugboat dispatch schedules, the underlying routing permutations constitute an NP-hard combinatorial problem. The state space of all possible inbound and outbound assignments is astronomically large, meaning an exhaustive, theoretically convergent search would exceed any reasonable time constraints for real-world maritime operations.

Consequently, the optimisation engine implemented in this digital twin deliberately sacrifices the guarantee of exhaustive finite convergence in favour of computational tractability and operational responsiveness. The system is designed as a truncated meta-heuristic, with the search space exploration strictly capped at a predefined limit of 100 iterations. By leveraging the Cheapest Insertion heuristic to establish a high-quality initial baseline, the Tabu Search efficiently navigates localised neighbourhoods and ensures that the simulator consistently yields robust, minimal risk dispatch schedules within the rapid timeframes necessary for ensuring timely tugboat service delivery.

5 Data Engineering Process

5.1 Dataset Overview

The project utilises Automatic Identification System (AIS) data to drive a data-driven "Log-play" simulation for a Maritime Digital Twin. The specific dataset analysed covers vessel movements in the Singapore Strait between January to December 2023. The raw data is segmented by month and day and stored in Comma Separated Values (CSV) zip files. Each file resides within its own folder of the same name. The total size of the dataset is 316 GB.

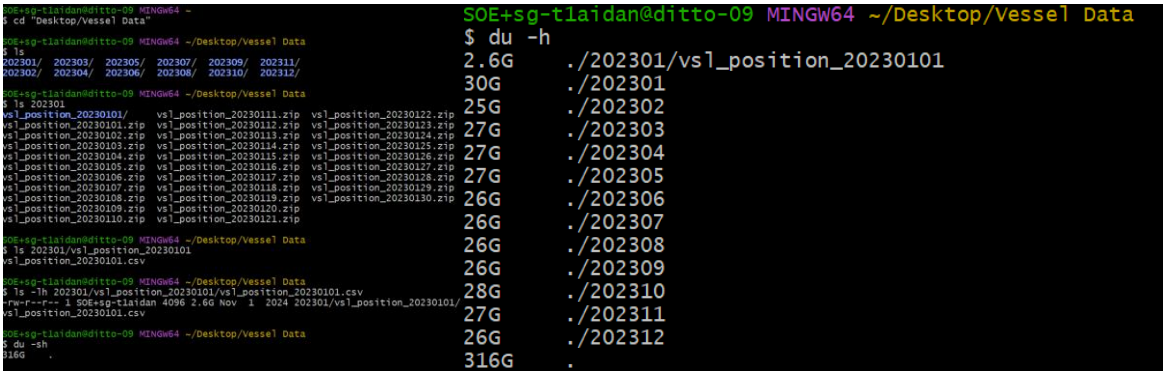

Figure 8: Overview of Vessel Movement Data Files

The raw data includes identifiers (mmsiNumber, imoNumber, name, callSign), vessel characteristics (type, length, grossTonnage, flag), and kinematic data (latitudeDegrees, longitudeDegrees, speedOverGround, headingOverGround, timeStamp).

5.2 Exploratory Data Analysis (EDA)

Exploratory data analysis of AIS data from 1 January 2023 was conducted to understand the traffic mix and operational behaviours required for the simulation. This file contained 19,701,668 records.

- **Missing / Null values:** About 19.67% of records had null values in the type column. Since we wish to distinguish unique vessel types, these were dropped from the dataset. Of the remaining records, 39.7% had an imoNumber of 0, and these were also removed

to ensure only legitimate containership and tugboat vessels were observed. 9,542,272 records remained after these two steps were performed, about 48.4% of this dataset.

- **Vessel Classification:** The raw AIS type codes were mapped to human-readable categories. The dominant vessel types identified were Tankers (TA), Tugs (TU), Container Ships (CS) and Bulk Carriers (CS).
- **Flag State Distribution:** Most vessels were flagged to Singapore (SG), Indonesia (ID), and Panama (PA). A data artifact was discovered where the flag code "01" appeared frequently; analysis revealed these were tugboats, tankers and surface vessels with unconfigured AIS transponders.
- **Operational Status:** Vessels were categorised by navigational status based on speed. Vessels traveling below 0.5 knots were classified as "At Anchor," while those above were "Under Way".

5.3 Vessel Length Prediction via Linear Regression

A critical requirement for the "Rule-Based Requirement Synthesis Engine" is the assignment of tugboats based on a vessel's Length Overall (LOA) in meters.

TUG ASSIGNMENT GUIDELINES FOR VESSELS BERTHING UNDER OWN POWER OR REQUIRED TO BE TOWED IN PORT

LOA (m) OF VESSEL	NUMBER AND SIZE OF TUGS FOR VESSEL WITH POWER	NUMBER AND SIZE OF TUGS FOR VESSEL WITHOUT POWER	TOWING TUG		ESCORT TUG	
			PILOTED MOVE	PILOTAGE EXEMPTED MOVE	PILOTED MOVE	PILOTAGE EXEMPTED MOVE
Up to 60m	1S	2S (Refer to Note 5)	Public Licence Towage Service Provider (PLTSP) tug only.	Need not be a PLTSP tug or ST prefix tug (Intra-port movement excluded)	PLTSP tug only	PLTSP tug or ST prefix tug
61 to 122m	2S	2S + 1S (Refer to Note 6)	PLTSP tug only	Need not be a PLTSP tug or ST prefix tug (Intra-port movement excluded)	PLTSP tug only	PLTSP tug or ST prefix tug
123 to 200m	2M	3M	PLTSP tug only		PLTSP tug only	
201 to 280m	2B	3B				
More than 280m or 75,000 GT and draft less than 17m	4B	5B				
More than 280m or 75,000 GT and draft 17m or more	4B (each with minimum bollard pull of at least 45 tonnes)	5B (each with minimum bollard pull of at least 45 tonnes)				
Cumbersome tow and vessels with unusual characteristics e.g. oil rigs, FPSO, semi-submersible, etc.	To be determine by Pilotage Service Provider in consultation with owner of vessel	To be determine by Pilotage Service Provider in consultation with owner of vessel				

Figure 9: Tug Assignment Guidelines by Maritime and Port Authority of Singapore

However, the training dataset from May to July 2023 contained corrupted length values (read as NaN or 0).

To resolve this, a linear regression model was trained using the 1 January 2023 dataset to predict a vessel's LOA based on its gross tonnage.

Since gross tonnage represents internal volume and volume scales cubically with length, the model regressed grossTonnage against the cube of a ship's length to achieve a linear fit. Separate models were trained for containerships and tankers due to their differing hull geometries.

Regression Model	Slope	Intercept	R^2 Score	Mean Absolute Error (meters)
Container Ships (CS)	334.97	4895.88	0.9575	10.91
Tankers (TA)	229.11	83941.03	0.9922	8.12

Table 1: Parameters and Metrics for LOA Linear Regression Model

The high accuracy achieved by both models confirms that grossTonnage is a reliable predictor for imputing missing length values, allowing the simulation to accurately assign tugboat resources even when AIS length data is corrupted.

5.4 Coupled Containership-Tugboat Trajectory Generation

To transition from raw historical data to a functional "Log-Play" simulation, a robust data processing pipeline was developed. This pipeline transforms unstructured Automatic Identification System (AIS) logs into discrete, validated trajectory assets representing coupled tug-vessel operations. The process follows a three-stage methodology: **Extraction**, **Coupling**, and **Validation**.

Stage I: Containership Manoeuvre Extraction

The primary challenge in processing the raw AIS dataset was its volume, which spanned multiple months and was segmented into daily compressed archives. A stream-processing

pipeline was implemented to extract relevant containership manoeuvres without exceeding memory constraints.

The pipeline first iterates through the data stream, monitoring the speed and position of unique vessels identified by their IMO number. A "Berthing Event" is flagged when a vessel's speed drops below **0.5 knots** while located within a designated **Berth Polygon** (defined via GeoJSON). Conversely, an "Unberthing Event" is flagged when a vessel accelerates from a stopped state within a berth.

To capture the full operational context, the extraction logic does not merely slice the data at the port boundary. Instead, upon detecting a docking event, the algorithm extracts a **90-minute historical lookback window**. This ensures the resulting trajectory includes the critical deceleration and tug rendezvous phases.

Raw AIS data frequently exhibits "jitter," where a docked vessel's speed fluctuates around the 0.5-knot threshold, triggering false positives. To address this, a **120-minute cooldown mechanism** was implemented, preventing the system from recording duplicate events for the same vessel within a two-hour window.

Stage II: Spatiotemporal Tug Coupling

Once the primary containership trajectories were isolated, the second stage involved identifying the specific tugboats that assisted each manoeuvre. Since no direct "job assignment" logs exist in AIS data, these relationships were inferred through a **Spatiotemporal Join** algorithm.

For each extracted containership trajectory, the algorithm scans the raw tugboat dataset for activity within the identical time window. Pandas' merge_asof logic was applied to align the asynchronous AIS pings of the tugs with the containership's timestamps, allowing for a tolerance of 30 seconds.

A tug was considered a candidate for coupling if it maintained a proximity of less than **200 meters** to the containership. To filter out unrelated traffic (e.g., tugs passing by in the channel), a **Duration Filter** was applied: a tug must maintain this proximity for at least **15 minutes** to be classified as an assisting vessel.

The pipeline effectively handles one-to-many relationships, identifying and grouping multiple tugs associated with a single Ultra Large Container Vessel (ULCV) job.

Stage III: Visual Validation and Asset Selection

The final stage focused on validating the physical realism of the extracted data before importing it into the Unity3D Digital Twin.

To verify the tugs' operational behaviour, trajectories were visualised using animated **Quiver Plots**. Unlike simple point markers, quiver plots display the **Heading** vectors of the vessels. This allowed for the visual distinction between "Towing" (tug vector pointing away from ship) and "Pushing" (tug vector perpendicular to ship hull) operations, confirming that the coupled entities were engaged in active assistance rather than coincidental movement.

The extracted dataset was subjected to manual review. Trajectories exhibiting significant AIS data gaps, signal jumps, or ambiguous coupling behaviour were discarded. Only the highest-fidelity trajectories—representing clear, continuous berthing manoeuvres with stable tug formations—were selected as the "Finalised Trajectories" to drive the Log-Play agents in the simulation.

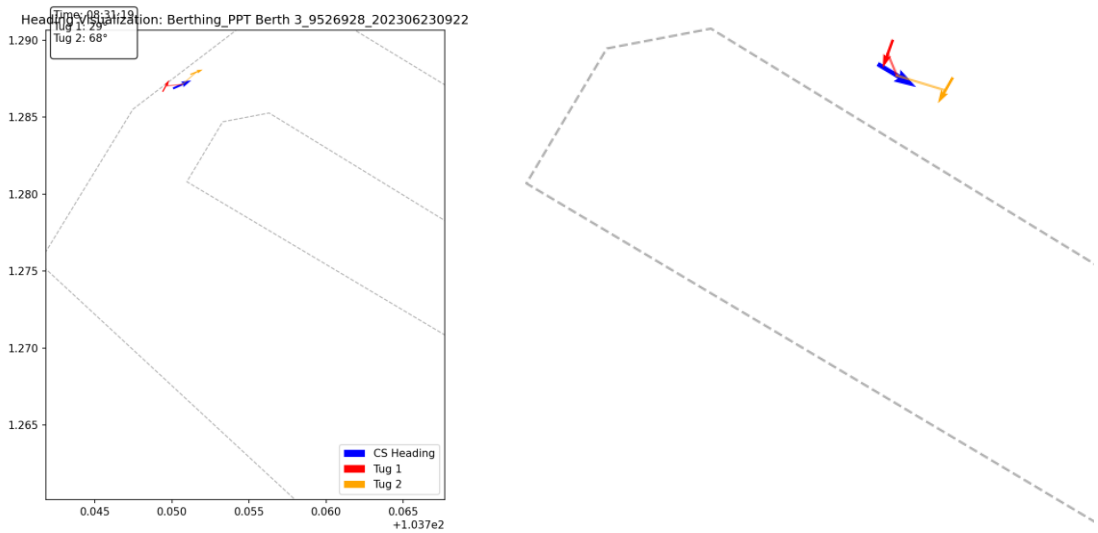

Figure 10: Quiver Plot Visualisations of Tugboat Operations

Left: Towing operation. Right: Pushing operation.

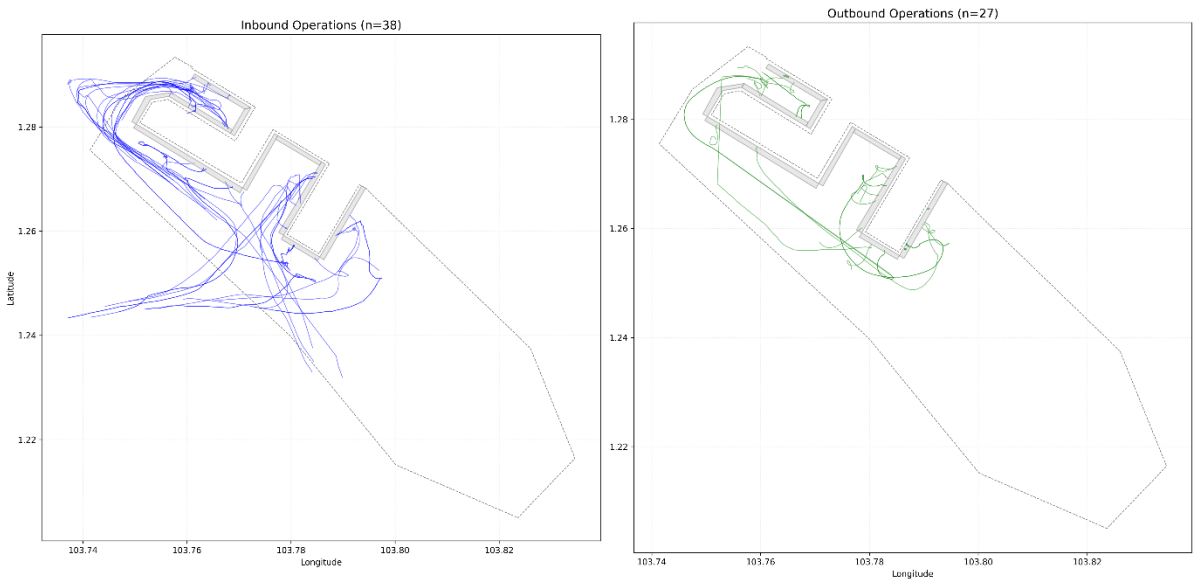

Figure 11: Visualisation of Extracted Containership Trajectories in Pasir Panjang Terminal

Left: Berthing trajectories. Right: Unberthing trajectories.

Hybrid Trajectory Generation for Large Vessel Operations

A significant challenge identified during data processing was the scarcity of complete 4-tug trajectory data in the AIS logs. To simulate high-fidelity operations for ULCVs, a hybrid approach combining historical data with procedural positioning was implemented.

Data-Driven Primary Tugs

For Large Vessel operations requiring 4 tugs, the first two tugs (Tug 1 and Tug 2) are driven by **historical AIS data**. These agents follow the precise, recorded trajectories extracted during the coupling phase.

Procedural Flanking Tugs

The remaining two tugs (Tug 3 and Tug 4) are procedurally positioned relative to the vessel's dynamic location to simulate flanking or braking assistance. These tugs are instantiated at fixed offsets to the port and starboard sides of the vessel's hull.

Unlike the data-driven tugs which have independent heading values, the procedural tugs align their heading to the vessel's current heading. This simulates the "towing" state where the tugs move in lockstep with the primary vessel to provide lateral stability or braking power.

Vessel Arrival Process and Visualising Tugboat Schedules

Containership vessel arrival schedules were simulated using an NHPP with the step function created in **Section 4.2.** The output of the traffic generation phase, as well as the input for the Unity3D visualisation, is a structured **JSON (JavaScript Object Notation)** file. This format serves as the data exchange layer between the Python-based traffic generator/optimiser and the C#-based simulation engine. To view the dispatch schedule for each tug, Python's built-in GUI library tkinter is used to pop up a window displaying the schedule.

Figure 12: Cheap Insertion tugboat allocation schedule generated for scenario4.

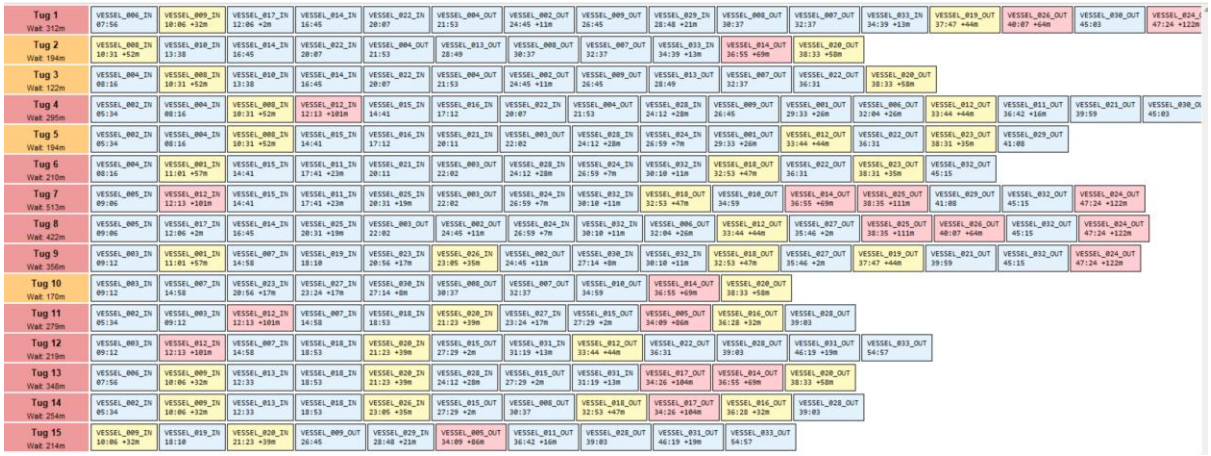

Figure 13: Cheap Insertion with Tabu Search tugboat allocation schedule generated for scenario4.

Jobs with predicted wait times exceeding 30 minutes are coloured yellow, and those exceeding 60 minutes are coloured red. Tugs whose schedules have total wait times exceeding 100 minutes are coloured amber, and those exceeding 200 minutes are coloured red.

These visualisations help to illustrate the intended effect of Tabu Search in optimising the schedules constructed by Cheap Insertion. Observe the decreased number of red jobs (30 in Figure 12 vs.. 20 in Figure 13) and the decreased number of red tug schedules (13 in Figure 12 vs.. 11 in Figure 13).

6 Simulation Engine Architecture

The core simulation engine is built within Unity3D, utilising a discrete-timestep architecture to bridge the gap between static schedules and dynamic 3D representation.

6.1 Asset Instantiation and Just-In-Time (JIT) Spawning

To maintain computational efficiency within the rolling horizon framework, the simulation employs a Just-In-Time (JIT) spawning mechanism. Two lists are maintained, one for the **fleet** of available tugboats and another for the pending **jobs** to be serviced. Rather than pre-loading all assets, a **JobManager** class monitors the global simulation time against the actualStartTime of pending **SimulationJobs.** At the start of each simulation run, **TugAgents** are instantiated at a **home base** located at coordinates (1.29, 103.76) in an Idle State.

Two types of containership vessel assets are used:

Small VesselAgent: Instantiated for jobs requiring 2 tugs or less (LOA < 280m).

Large VesselAgent: Instantiated for jobs requiring > 2 tugs (typically ULCVs requiring 4 tugs with LOA > 280m).

Coordinate Synchronisation: Vessels are instantiated at the precise (latitude, longitude) coordinates of the first timestamp in their assigned trajectory file.

6.2 Finite State Machine (FSM) Navigation

Tug behaviour is governed by a Finite State Machine with three distinct modes:

1. **Mobilising (Autonomous):** The tug calculates an A* path on the NavMesh from its current position to the target vessel's rendezvous point. It dynamically avoids static obstacles defined in the bake.
2. **Towing (Coupled):** Upon entering a proximity threshold (< 25m) of the target, the NavMeshAgent component is disabled. The tug enters a "Coupled" state, where its

transform is driven directly by the target vessel's trajectory updates (either historical or procedural).

3. **Idle (In-Situ):** Upon job completion (berthing or release), the tug transitions to an idle state. Idle tugs can either return to their home base or transit to the location of their next assignment, depending on the assignment policy employed.

6.3 Tug Assignment

Once a vessel is instantiated, the JobManager executes a resource assignment routine tailored to the active scheduling protocol. The process begins by filtering the fleet list for tugs whose CurrentState is Idle.

The selection logic then diverges based on the scheduling mode. In **reactive scheduling,** tugs are only assigned upon vessel arrival. Two policies are available:

- **FCFS Policy:** A baseline where tugs are dispatched to service containerships in the order of their arrival. This simulates a rigid port queue, resulting in Head-of-Line (HOL) blocking when resources are insufficient for the leading vessel.
- **Greedy Policy:** The system performs a spatial query, calculating the Euclidean distance from the vessel's spawn point to every available idle tug. The algorithm dynamically selects the set of tugs that minimises immediate mobilisation distances. Tugs can be assigned to any waiting vessel that meets resource requirements, allowing for queue jumping to maximise fleet throughput.

In **proactive scheduling,** tugs are pre-allocated by an optimisation algorithm and can thus be prepositioned at the location of their next pending job upon completion of service. The two algorithms available are **CI and CI-TS,** as covered in **Section 4.4.**

In all configurations, the assignment is strictly conditional: the job proceeds only if the required candidate tugs (whether nearest or pre-assigned) are verified as Idle. Upon successful

allocation, the selected tugs are locked for the specific job ID, and their operational state is immediately transitioned to Mobilising.

6.4 Navigation and Pathfinding

The simulation environment constructs a navigational graph that represents the specific topology of the Pasir Panjang Terminal.

NavMesh Baking

Unity's AI navigation stack is adapted to the maritime domain:

- **Static Geometry:** Landmasses, breakwaters, and wharves are marked as Navigation Static and Not Walkable.
- **Walkable Surface:** The sea surface is baked into a specific NavMesh layer.
- **Agent Radius:** The bake settings use a radius of 10 meters, approximating the width of a standard harbour tug. This ensures that autonomous tugs naturally avoid collisions with static port infrastructure during their mobilisation and return phases.

6.5 Measuring Performance Metrics

The effectiveness of the Digital Twin is measured by comparing the performance of the Baseline Simulation (using FCFS logic) against the Optimised Simulation (using Tabu Search).

Primary Metric: Average Wait Time Reduction

The system logs the timestamp of every state change. The wait time for each vessel is calculated as the delta between its ETA and the moment the final required tug arrives (serviceStartTime). The cumulative sum is taken and averaged across all vessels.

A reduction in average wait time indicates that the Tabu Search successfully identified "clusters" of jobs and allocated tugs effectively, reducing the idle time caused by deadheading.

Secondary Metric: Safety and Near-Miss Count

To detect "Near-Miss" events, we make use of Unity's Physics Engine, following the framework below:

- **Trigger Detection:** A simplified collider (Trigger Mode) representing a 3-cable safety radius (approx. 555m) is attached to every active vessel.
- **Event Logging:** If a tug or vessel enters this trigger zone during a non-towing phase, a "Near-Miss" event is logged.

An optimised schedule should not have a near-miss count that statistically exceeds that of the baseline, ensuring that efficiency gains are not achieved by compromising the 3-cable safety buffer.

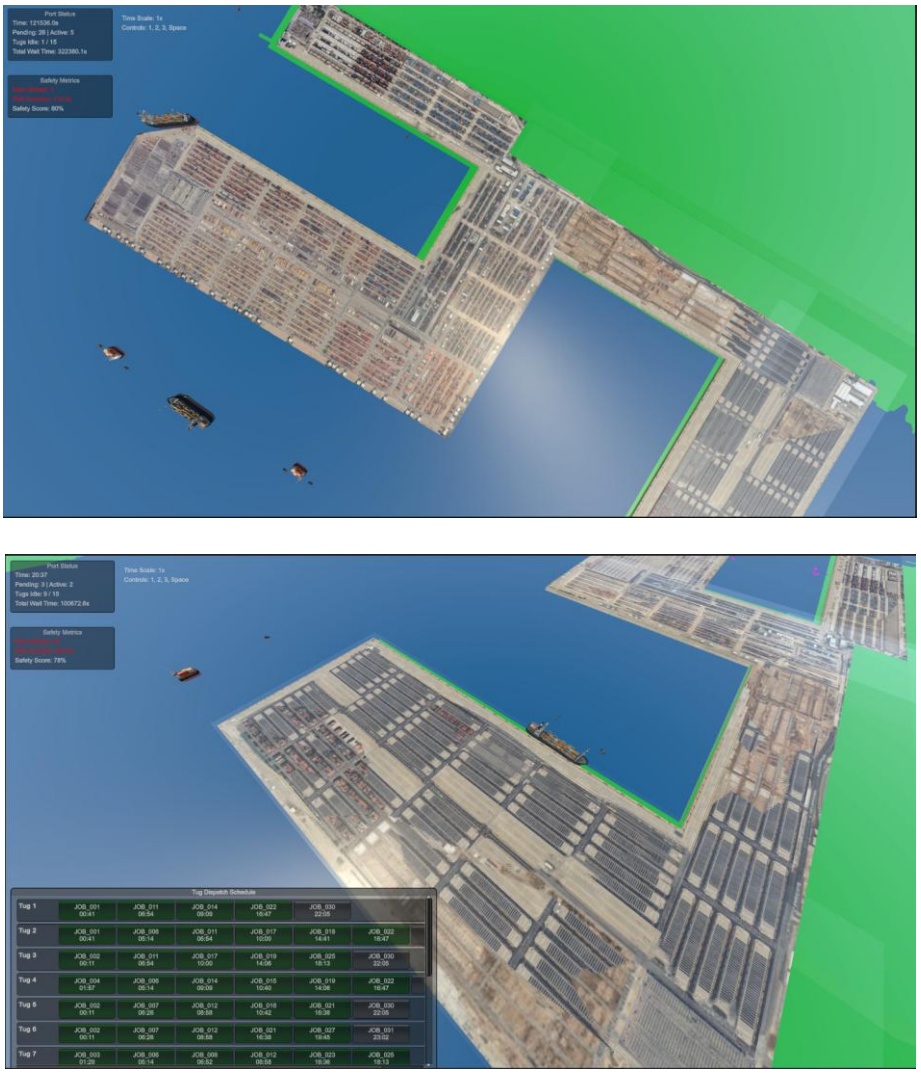

Figure 14: Unity3D Simulation with GUI for monitoring performance metrics.

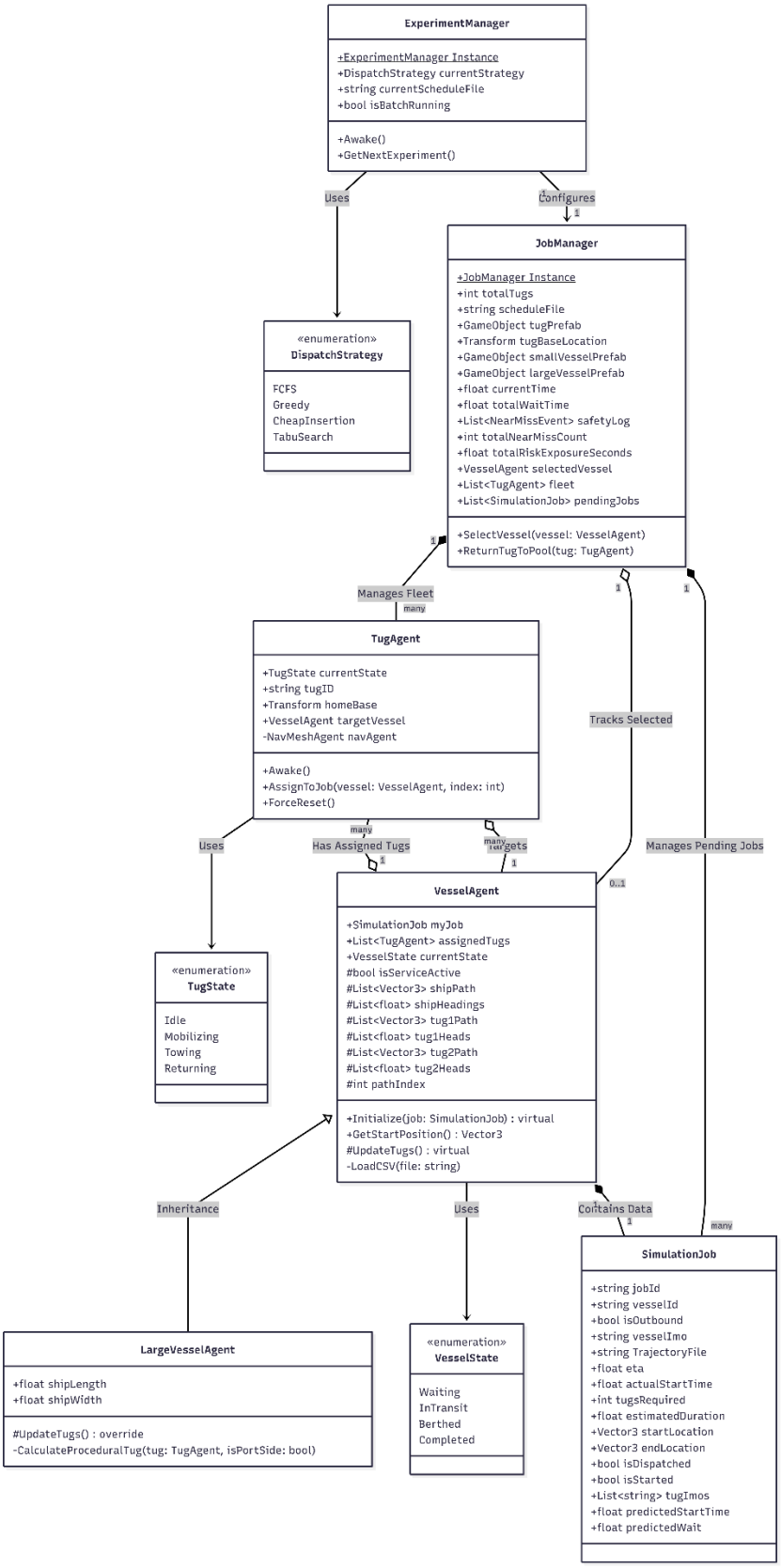

Figure 15: UML Diagram of main classes in Unity3D Simulation Architecture

7 Experiments and Results

A batch-processing experimental framework was designed to evaluate the efficacy of the proposed scheduling algorithms. The objective of this experiment is to benchmark the performance of the CI heuristic and TS metaheuristic approach against a standard operational baseline within a highly constrained, dynamic digital twin environment.

7.1 Simulation Scenarios

To ensure statistical significance and avoid overfitting to a single traffic pattern, 31 unique daily traffic scenarios were generated. Vessel arrivals were modelled using a Poisson distribution based on historically derived arrival rates for Pasir Panjang Terminal. To maintain perfect reproducibility while ensuring dataset variance, a dynamic seeding mechanism was utilised across the 31 iterations.

7.2 Evaluated Scheduling Policies and Hypothesis

The experiment measures the performance of the four scheduling policies: **FCFS, Greedy, CI,** and **CI-TS.**

The primary hypothesis (H_1) posits that the **proactive scheduling policies will outperform the reactive scheduling policies,** either by reducing overall vessel wait times or improving overall safety score. The null hypothesis (H_0) assumes **no statistically significant difference in performance metrics between the operational strategies.**

7.3 Evaluation Metrics

To test this hypothesis, the digital twin simulation measures performance across two primary dimensions:

- **Operational Efficiency:** Measured by the **Mean Average Wait Time** (in hours). This calculates the average difference between a vessel's ETA and the actual time service commences once all required tugs have arrived.

- **Navigational Safety:** Measured by the **average count** of near-miss events, and **Mean Safety Score** expressed as a percentage.

The formulas for both metrics and safety radius are covered in **Sections 3.2 and 3.3.**

7.4 Testing Methodology

The 31 pre-computed daily scenarios are fed into the Unity digital twin via an automated **ExperimentManager.** The simulation runs batch executions at an accelerated 50x speed timescale with 15 available tugs, sequentially applying the four scheduling policies to each of the 31 scenarios. yielding 124 total simulation runs. Upon the completion of all jobs in each scenario, the ExperimentManager logs the efficiency and safety metrics to a centralised CSV dataset. Preliminary analysis of the dataset revealed unequal variances (heteroscedasticity) in wait times across the algorithms, a direct result of the proactive optimisation policies intentionally inducing a wider spread of individual delays to satisfy spatiotemporal safety constraints. To mathematically account for this violation of the equal-variance assumption, the normalised dataset was subjected to Welch's t-tests rather than standard independent t-tests. This approach ensured a robust determination of whether the comparative differences in efficiency and safety metrics between the reactive and proactive scheduling policies achieved true statistical significance.

7.5 Results

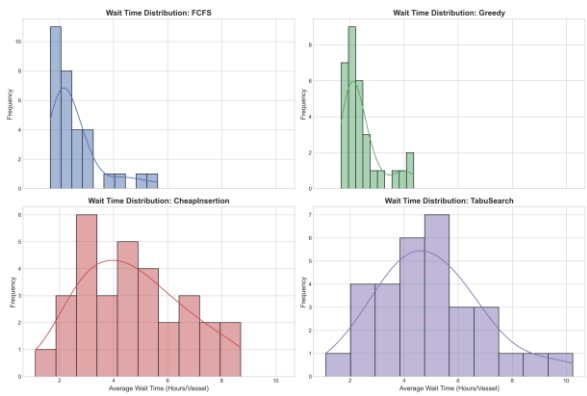

Figure 16: Histogram Distributions of Average Wait Times

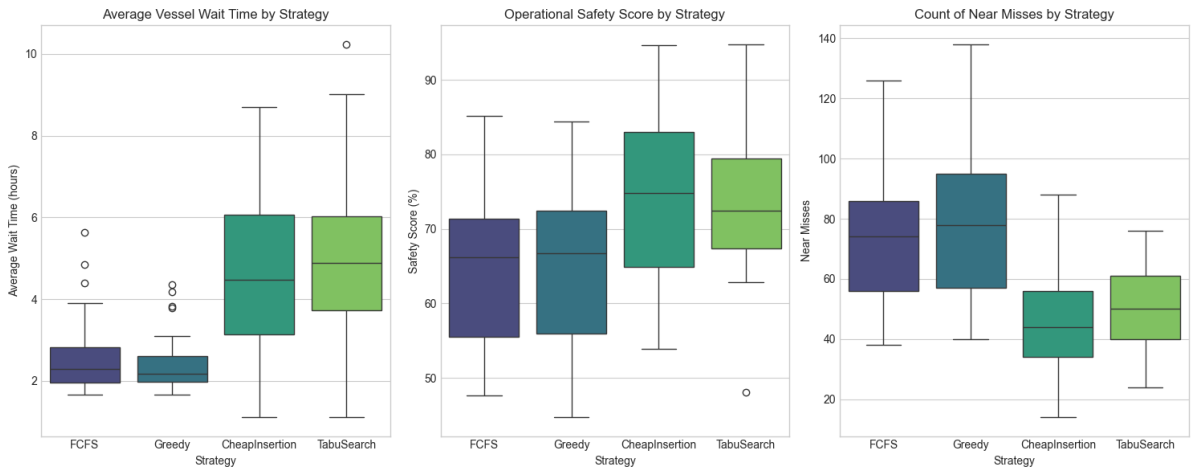

Figure 17: Boxplots of Key Performance Metrics

Strategy	**Mean ± Std. Dev**		
	Wait Time (Hrs/Vessel)	Safety Score (%)	Near Misses
Greedy	**2.43 ± 0.71**	64.99 ± 9.67	77.94 ± 25.78
FCFS (Baseline)	2.57 ± 0.94	64.50 ± 9.89	74.26 ± 24.50
Cheap Insertion (CI)	4.72 ± 1.92	**73.99 ±** 11.22	**47.61 ±** 18.12
Tabu Search (CI-TS)	4.98 ± 2.00	73.81 ± **9.92**	49.81 ± **14.68**

Table 2: Summary Statistics of Key Performance Metrics

Average Wait Time Comparison	**t-statistic**	**p-value**	**Significance**
FCFS vs. Cheap Insertion	-5.5846	< 0.0001	Highly Significant
FCFS vs. Tabu Search	-6.0505	< 0.0001	Highly Significant
Greedy vs. Cheap Insertion	-6.2282	< 0.0001	Highly Significant
Greedy vs. Tabu Search	-6.6824	< 0.0001	Highly Significant

Table 3: ANOVA Hypothesis Testing Results for Average Wait Time

Mean Safety Score Comparison	t-statistic	p-value	Significance
FCFS vs. Cheap Insertion	-3.5319	0.0008	Highly Significant
FCFS vs. Tabu Search	-3.6983	0.0005	Highly Significant
Greedy vs. Cheap Insertion	-3.3843	0.0013	Highly Significant
Greedy vs. Tabu Search	-3.5448	0.0008	Highly Significant

Table 4: ANOVA Hypothesis Testing Results for Mean Safety Score

Avg. Near Misses Comparison	t-statistic	p-value	Significance
FCFS vs. Cheap Insertion	4.8677	8.59×10^{-6}	Highly Significant
FCFS vs. Tabu Search	4.7658	1.24×10^{-5}	Highly Significant
Greedy vs. Cheap Insertion	5.3575	1.41×10^{-6}	Highly Significant
Greedy vs. Tabu Search	5.2788	1.89×10^{-6}	Highly Significant

Table 5: ANOVA Hypothesis Testing Results for Average Near Misses

7.6 Discussion

Reactive policies like **Greedy** and **FCFS** provide the lowest wait times, averaging **2.43** and **2.57 hours** per vessel respectively. On the other hand, proactive policies such as **CI** and **CI-TS** offer superior risk mitigation, achieving the highest Safety Scores of **74%** compared to 65% for the reactive policies. **CI** recorded the lowest mean near misses (**47.61**), compared to **77.94** for the Greedy strategy. This comes at the expense of wait times that are up to **twice as long** compared to the reactive policies.

The standard deviation for wait times is higher in optimised strategies, suggesting that safety-first scheduling is more sensitive to high-congestion traffic spikes. Notably, **CI-TS** maintained a more consistent safety performance than **CI**, with smaller standard deviations in both Near Misses (14.68 VS 18.12) and Safety Score (9.92 VS 11.22) proving its reliability in maintaining the required safety buffers despite varying vessel traffic scenarios.

Turning to the t-tests between scheduling policies, comparisons for **Average Wait Time** yielded extremely low p-values, confirming that the safety gains introduced by the optimisation algorithms come with a consistent operational cost. Safety score improvements and near miss reductions are equally robust, as proactive scheduling policies outperform reactive policies with p-values less than 0.002.

Although the comparative performance trends among the evaluated scheduling policies are conclusive, a discrepancy exists regarding the absolute values of the recorded wait times. In empirical maritime operations, tug dispatch delays typically range from 5 to 45 minutes; however, the simulation yielded average wait times of 2 to 5 hours. This inflation stems from a structural constraint within the digital twin's architecture, where the core simulation logic is currently coupled with the graphical frame rate. Processing an entire month of traffic at a highly accelerated timescale introduces severe computational overhead from the 3D visual rendering. This rendering bottleneck induces a temporal lag in both vessel kinematics and metric logging, artificially inflating the absolute wait times and risk exposure durations. Nevertheless, because this systemic latency applies uniformly across all experimental runs, the relative performance differentials, such as the superior safety consistency achieved by Tabu Search, remain analytically robust. To resolve this discrepancy in future iterations, the physical calculation engine must be entirely decoupled from the graphical rendering pipeline, allowing the simulation to execute asynchronously or in a 'headless' state to generate precise, real-world metrics.

In summary, the results demonstrate a statistically significant trade-off between efficiency and risk. While reactive policies minimise vessel wait times, proactive algorithms. particularly CI-TS, sacrifice speed to deliver superior and highly consistent safety performance, even during traffic spikes.

8 Conclusion and Future Work

The primary objective of this project was to develop a high-fidelity Digital Twin Simulator to evaluate and optimise tugboat scheduling policies. By pairing a Unity3D-based simulation environment with Python-driven algorithmic optimisation, the system successfully modelled complex, multi-agent maritime operations.

The evaluation of scheduling policies revealed a statistically significant trade-off between operational throughput and maritime safety. While reactive policies like FCFS and Greedy scheduling minimised average vessel wait times, they struggled to maintain spatial safety constraints during high-congestion periods. Conversely, proactive policies like CI-TS demonstrated superior and highly consistent risk mitigation. CI-TS successfully prioritised the 555-meter safety buffer, mitigating near-miss events even during traffic spikes, albeit at the cost of higher average waiting times. These findings provide actionable insights for port authorities such as the Maritime and Port Authority of Singapore (MPA) when designing dispatch protocols that must dynamically balance port efficiency against strict navigational safety standards.

8.1 Codebase

The codebase of this project is organised into the following components:

- **optimisation/:** This directory houses the route-planning and optimisation algorithms (Cheapest Insertion and Tabu Search). It processes the generated containership arrival schedules to generate optimised tugboat dispatch schedules.
- **simulation/:** Integrated as a Git submodule, this repository contains the Unity3D digital twin. Driven by C# scripts, it dynamically generates baseline tugboat schedules using First-Come-First-Serve and Greedy approaches within the JobManager class, and it

ingests the Python-optimised JSON schedules to execute and validate them within a deterministic, physics-based spatial environment.

- **Preprocessing (data_preparation/, Pasir Panjang Terminal/, trajectories/, jobs/):** These directories compartmentalise the various static data inputs required by the optimisation engine, port zones and boundaries, vessel movement histories, and coupled job definitions from the active scripts.
- **Outputs (generated_schedules/, results/):** The generated_schedules/ stores the containership arrival and tugboat dispatch schedules in JSON format, ready to be read by Unity. Post-simulation, the results/ directory holds the raw performance metrics for statistical analysis.
- **docs/**: This directory stores the visual and explanatory media, housing the final report assets, architecture diagrams, and the project demo video.

The codebase is accessible at the following GitHub repository: https://github.com/ardentaegis17/TugboatSchedulingDigitalTwinSimulator

8.2 Future Improvements

Building upon the foundation of this digital twin, future iterations of the system should prioritise enhancing both operational realism and computational accuracy.

While the current iteration of the simulator successfully demonstrates the efficacy of the job-centric Tabu Search algorithm, its predictive capabilities are inherently bounded by its reliance on offline, historical AIS data. To evolve this architecture from a simulation model into a fully mature Digital Twin, future development must prioritise the integration of live data streams.

Subsequent iterations should explore establishing continuous, automated telemetry with live maritime data platforms, such as MPA's OCEANS-X API. Ingesting real-time data such as live vessel coordinates, dynamic ETA updates, localised weather conditions, and active berth availability would allow the engine to transition from offline schedule generation to online, dynamic re-optimisation. By feeding live state-changes directly into the Unity3D environment and the Python optimisation engine, the system could continuously recalculate the Tabu Search heuristics on the fly. This would help support a real-time operational dashboard, empowering terminal dispatchers with immediate, mathematically optimised decision support to dynamically reallocate tugboats during unexpected maritime delays or port congestion events.

To ensure the simulator scales effectively for extensive batch experimentation, subsequent architectural improvements should also address the computational limitations of the visual rendering pipeline. Running the simulation engine at accelerated time scales can currently lead to frame-skipping and floating-point inaccuracies inherent in visual frame-rate calculations, which introduces discrepancies when logging time-dependent metrics such as waiting times and risk exposure durations. Transitioning to a fixed-tick, headless simulation core would decouple the metric-gathering logic from the visual rendering loop. This adjustment would guarantee absolute deterministic accuracy across all batch experiments, regardless of execution speed, thereby fortifying the statistical reliability of the collected data.

A third future enhancement to the simulator can include integrating native, physics-based collision tracking. Rather than relying on simplified static-radius safety buffers which overstate lateral risk for elongated mega-vessels, future architectures should implement geometry-conforming colliders. By utilising dynamic BoxCollider or convex MeshCollider components scaled precisely to each vessel's specific Length Overall (LOA) and Beam, the simulation can enforce anisotropic safety margins. This allows the digital twin to track physical boundary breaches that accurately reflect the unique spatial footprint of the ships. Implementing these precise geometric triggers would provide a highly granular empirical metric to validate the heuristic safety scores, ensuring that the algorithm's performance is grounded in true kinematic reality rather than generalised proximity.

8.3 Acknowledgements

I would like to express my deepest appreciation to my academic supervisor, Prof. Tong Xin, for his invaluable guidance throughout this final year project. His expertise and meticulous feedback were instrumental in establishing the mathematical rigor of this work, particularly in refining the formulation of the optimisation algorithms.

I am equally grateful to my company supervisor, Lawrence Chew, for his continuous mentorship. His evaluation and support were essential in bridging the gap between the proposed theoretical models and a robust, realistic simulation engine.

Finally, I would like to extend my sincere thanks to the Maritime and Port Authority of Singapore (MPA). Their generous provision of historical AIS data and their willingness to support this initiative provided the exceptional opportunity to develop and validate this digital twin.

8.4 Declaration of Generative AI Usage

During the development of this project, Large Language Models (LLMs) were utilised as an auxiliary tool to assist with code debugging, structural formatting, and technical documentation. Specifically, Google's Gemini was employed to aid in the identification and resolution of algorithmic logic flaws, particularly in refining the neighbourhood exploration and Tabu List tracking mechanisms within the Python optimisation engine. Furthermore, the model assisted in drafting and structuring repository documentation, including the formatting for the Unity simulation environment and the generation of the UML diagram for the simulation engine's C# component classes. In the research phase, the AI aided in the literature synthesis by helping to articulate the methodological gaps between existing First-Come, First-Served (FCFS) simulation frameworks and dynamic multi-tug allocation models. Finally, it provided code troubleshooting support to resolve library-specific compilation and rendering errors, such as bypassing strict tracking prevention and CORS restrictions in Python-generated geographic map visualisations.

All AI-generated suggestions were manually reviewed, tested, and integrated by the author to ensure empirical accuracy and alignment with the project's architectural constraints.

8.5 Bibliography

Antil, H. (2024). Mathematical Opportunities in Digital Twins (MATH-DT). ArXiv.org. https://arxiv.org/abs/2402.10326

Glover, F., & HanafiS. (2002). Tabu search and finite convergence. *Discrete Applied Mathematics*, *119*(1-2), 3–36. https://doi.org/10.1016/s0166-218x(01)00263-3

Google. (2026). *Gemini 3.1 Pro* (April 9 version) [Large language model]. https://gemini.google.com

Ilati, G., Sheikholeslami, A., & Hassannayebi, E. (2014). A Simulation-Based Optimization Approach for Integrated Port Resource Allocation Problem. *PROMET - Traffic&Transportation*, *26*(3), 243–255. https://doi.org/10.7307/ptt.v26i3.1337

Manickam, S., Yarlagadda, L., Gopalan, S. P., & Chowdhary, C. L. (2023). Unlocking the Potential of Digital Twins: A Comprehensive Review of Concepts, Frameworks, and Industrial Applications. IEEE Access, 11, 135147–135158. https://doi.org/10.1109/access.2023.3338530

Nagrani, S., Gunasekaran, A., & Narwane, V. S. (2025). Enabling Digital Twins: a simulation software perspective. Digital Twin. https://doi.org/10.1080/27525783.2025.2533848

Paulius Skaisgiris, Simoncini, W., Barbero, F., Amir Ahangi, & Mockel, R. (2021). PySeidon - A Data-Driven Maritime Port Simulation Framework. *Data Archiving and Networked Services (DANS)*. https://doi.org/10.1145/3474963.3474986

Quang, A., Pham, Chathuranga Brahmanage, J., Kumar, A., Pham, J., Brahmanage, A., & Kumar. (2025). ShipNaviSim: Data-Driven Simulation for Real-World Maritime Navigation. *IFAAMAS*, *9*, 2025. https://shipnavisim.github.io/static/ShipNaviSim-camera-ready.pdf

TUG ASSIGNMENT GUIDELINES FOR VESSELS REQUIRED TO BE TOWED IN PORT (n.d.). Retrieved March 26, 2026, from https://www.mpa.gov.sg/docs/mpalibraries/mpa-documents-files/oms/ops-pilotage-and-planning/tug_assignment_guidelines.pdf

Wei, X., Jia, S., Meng, Q., and Koh, J. (2024). Dynamic tugboat deployment and scheduling with stochastic and time-varying service demands. Transportation Res. Part B-Methodological 188, 103059. https://doi.org/10.1016/j.trb.2024.103059

Wong, H.-L., Hsieh, S.-H., & Tu, Y.-H. (2009). Application of Non-Homogeneous Poisson Process Modeling to Containership Arrival Rate. *2009 Fourth International Conference on Innovative Computing, Information and Control (ICICIC)*, 849–854. https://doi.org/10.1109/icicic.2009.111

Zhang, Y. (2024). Digital Twin Architectures, Networks & Applications. Springer.

8.6 Changelog

Sections Modified: Chapters: Experiments and Results (Subsections: Testing Methodology, Discussion), Conclusion and Future Work (Subsection: Future Improvements)

Updates Made:

- **Added:** A theoretical framework detailing the future integration of empirical collision and near-miss tracking within the digital twin environment. The use of Welch's t-tests to account for unequal variances in performance metrics across the four scheduling strategies and the discrepancy in the absolute values of average wait times due to visual rendering performance bottlenecks are mentioned explicitly.
- **Refined Spatial Logic:** Replaced the concept of static, circular safety buffers with geometry-conforming boundary tracking. Added an explanation of how utilising dynamically scaled bounding boxes (matching the vessel's specific length and beam) eliminates the false-positive overlaps inherent in radial collision detection for elongated ships.

Appendix A: RoutePlanner

Overview

The RoutePlanner class provides essential spatial awareness to the scheduling engine by calculating the shortest navigable paths for tugboats around physical port infrastructure. Rather than relying on simple Euclidean distances, which may incorrectly suggest paths that intersect with landmasses, this class ensures that all calculated travel times reflect physically viable maritime routes.

Coordinate Transformation

To maintain consistency between the offline Python optimisation engine and the Unity3D digital twin, the RoutePlanner must map geographic coordinates to a local 2D plane.

It accepts a reference origin point (latitude and longitude) that corresponds to the (0,0) origin in the Unity environment and utilises an Equirectangular projection to convert the geographic coordinates of the terminal boundaries into a Cartesian coordinate system measured in meters.

Obstacle Avoidance via Visibility Graphs

The core routing logic utilises a visibility graph approach to navigate around the terminal polygon. The class first loads the physical boundary of the terminal from a GeoJSON file and applies a 50-meter spatial buffer using the shapely library. This buffer acts as a safety margin to prevent calculated routes from scraping the quay wall.

When calculating the distance between two points (start_pos and end_pos), the system performs a line-of-sight check. If a direct line connecting the two points does not intersect the buffered terminal polygon, the simple Euclidean length is returned.

If the direct path is obstructed, the class constructs a spatial network graph using the networkx library. Edges are generated to connect the start and end points to any visible corners of the buffered polygon, and additional edges connect the adjacent corners along the polygon's

perimeter. Dijkstra's algorithm is then applied to this graph to determine the shortest path length that safely navigates around the obstacle.

Integration with the Insertion Heuristic

The realistic distances calculated by the RoutePlanner are a critical component when evaluating the costs of schedules generated by the Cheap Insertion and Tabu Search algorithms.

When first loading the algorithms, the planner is instantiated globally as router. When finding resource availability, the algorithm evaluates which tugboats are best positioned to serve an incoming job.

For every available tugboat, the script calls router.get_safe_distance() to determine the navigable distance between the tug's current location and the job's starting coordinates. This navigable distance is divided by the standard tugboat speed (TUG_SPEED_MS) to calculate an accurate travel time.

Consequently, the algorithm generates a dynamically feasible schedule where a tug's estimated time of arrival (ready_at) strictly accounts for the physical geography of the Pasir Panjang Terminal.